

Bayesian Networks: Inference using Variable Elimination

Introduction

In the realm of artificial intelligence, understanding and predicting outcomes in uncertain environments is paramount. Bayesian networks provide a powerful framework for modeling probabilistic relationships between variables. After establishing the syntax and semantics—how these networks are structured and what probabilistic statements they represent—the next crucial step is inference. Inference in Bayesian networks is akin to asking "what if" questions: given some observed evidence, what is the probability of other events or states? This process allows us to update our beliefs and make informed decisions based on partial information.

The core challenge lies in efficiently computing these probabilities, especially when dealing with many interconnected variables. A naive approach of considering all possible joint probabilities quickly becomes intractable. This lecture introduces Variable Elimination, a fundamental algorithm designed to perform exact inference in Bayesian networks by systematically summing out (eliminating) hidden variables, leveraging the network's graphical structure to optimize computations. We will explore this through a practical example, calculating the probability of a burglary given that both John and Mary have called.

Core Concepts

Term	Definition	Significance
Inference	The process of determining the probability of query variables given observed evidence in a probabilistic model.	Allows for updating beliefs and making predictions based on partial information, central to AI applications.
Marginal Probability	The probability of a single variable, calculated by summing out all other variables from a joint distribution, without any given evidence.	Represents the standalone likelihood of an event in the context of the full model, useful for baseline understanding.

Term	Definition	Significance
Posterior Probability	The probability of a variable or event after accounting for some observed evidence.	Crucial for decision-making, as it reflects updated beliefs based on new information.
Hidden Variables	Variables in a Bayesian network whose values are not observed and are not the primary query, but influence the relationships between observed and query variables.	Must be accounted for (summed out) during inference to correctly compute probabilities of interest.
Factor	A function that maps a configuration of a subset of variables to a numerical value, often representing a conditional probability table (CPT).	The fundamental building block in variable elimination; operations are performed on these factors.
Variable Elimination	An exact inference algorithm for Bayesian networks that systematically sums out hidden variables by manipulating and multiplying factors.	Optimizes inference by avoiding the explicit construction of the full joint distribution, reducing computational redundancy.
Dynamic Programming	An algorithmic technique that solves complex problems by breaking them into simpler subproblems and storing the results of these subproblems to avoid recomputing them.	The underlying principle of variable elimination, where intermediate factors are stored and reused.
NP-hard	A class of computational problems for which finding an optimal solution is at least as hard as the hardest problems in NP (Non-deterministic Polynomial time).	Describes the complexity of finding the optimal variable elimination order, indicating that heuristics are often used in practice.
#P-complete	A complexity class for counting problems, which are generally	Bayesian network inference can be used to solve #P-

Term	Definition	Significance
	harder than their decision problem counterparts (e.g., counting solutions instead of just finding one).	complete problems like model counting, demonstrating its expressive power.

Detailed Analysis

1. The Inference Problem and Factorization

The primary goal of inference in Bayesian networks is to compute the posterior probability of a query variable (Z) given some evidence (E), denoted as $P(Z \mid E)$. This is a conditional probability, which by definition can be expressed as $P(Z, E) / P(E)$.

A key simplification in this calculation is recognizing that the denominator, $P(E)$, is a constant with respect to the query variable Z . Therefore, $P(Z \mid E)$ is proportional to $P(Z, E)$. This means we can compute $P(Z, E)$ for all possible values of Z , and then normalize these results by dividing by their sum to obtain the true posterior probabilities. This proportionality trick significantly streamlines the computation.

To compute $P(Z, E)$, we need to consider all other variables in the network that are neither query nor evidence variables; these are the hidden variables. The process involves summing out these hidden variables from the full joint probability distribution. For a network with variables $X_1, \dots, X_n$, the full joint distribution $P(X_1, \dots, X_n)$ can be factorized according to the Bayesian network's structure as a product of conditional probability tables (CPTs):

$$P(X_1, \dots, X_n) = \prod P(X_i \mid \text{Parents}(X_i))$$

For our example, calculating the probability of burglary (B) given John called (j) and Mary called (m), we are interested in $P(B \mid j, m)$. This is proportional to $P(B, j, m)$. To find $P(B, j, m)$, we must sum over the hidden variables, which in this case are Earthquake (E) and Alarm (A):

$$P(B, j, m) = \sum_E \sum_A P(B, E, A, j, m)$$

Using the factorization rule for the Bayesian network, the full joint probability $P(B, E, A, j, m)$ can be written as:

$$P(B) * P(E) * P(A \mid B, E) * P(j \mid A) * P(m \mid A)$$

Thus, the problem transforms into computing:

$$P(B, j, m) \propto \sum_E \sum_A [P(B) * P(E) * P(A \mid B, E) * P(j \mid A) * P(m \mid A)]$$

A naive summation over all 2^N states (where N is the number of variables) would be computationally expensive. Variable elimination optimizes this by pushing summations inward and performing computations on smaller "factors" (intermediate

results) rather than the entire joint distribution. This approach leverages the principle of dynamic programming, avoiding redundant calculations.

2. Variable Elimination: The Algorithm in Action

The variable elimination algorithm proceeds by iteratively eliminating hidden variables. Each elimination step involves two main operations:

1. **Joining factors:** Multiplying all factors that contain the variable to be eliminated.
2. **Summing out the variable:** Summing over all possible values of the variable from the resulting product.

Let's trace the variable elimination process for $P(B \mid j, m)$ using the example:

$$P(B \mid j, m) \propto \sum_E \sum_A [P(B) * P(E) * P(A \mid B, E) * P(j \mid A) * P(m \mid A)]$$

The known evidence variables 'j' and 'm' are treated as fixed values (lowercase 'j' and 'm'). The uppercase variables (B, E, A) represent random variables that can take on different values (e.g., true/false).

Step 1: Initial Factors and Eliminating 'j' and 'm' (implicitly)

The initial factors are the CPTs: $P(B)$, $P(E)$, $P(A \mid B, E)$, $P(j \mid A)$, $P(m \mid A)$.

Since 'j' and 'm' are evidence, their values are fixed. We can combine $P(j \mid A)$ and $P(m \mid A)$ into a new factor, $f1(A)$, which is a function of A.

$$\text{For } A=\text{true}: f1(\text{true}) = P(j=\text{true} \mid A=\text{true}) * P(m=\text{true} \mid A=\text{true})$$

$$\text{For } A=\text{false}: f1(\text{false}) = P(j=\text{true} \mid A=\text{false}) * P(m=\text{true} \mid A=\text{false})$$

For instance, if $P(j \mid A=\text{true}) = 0.9$ and $P(m \mid A=\text{true}) = 0.7$, then $f1(\text{true}) = 0.9 * 0.7 = 0.63$.

If $P(j \mid A=\text{false}) = 0.05$ and $P(m \mid A=\text{false}) = 0.01$, then $f1(\text{false}) = 0.05 * 0.01 = 0.0005$.

This effectively "eliminates" the variables 'j' and 'm' from further explicit consideration by incorporating their evidence into a factor over 'A'.

Step 2: Eliminating 'A'

Next, we eliminate the hidden variable 'A'. We identify all factors that contain 'A': $P(A \mid B, E)$ and $f1(A)$.

We join these factors by multiplying them: $P(A \mid B, E) * f1(A)$.

Then, we sum out 'A' from this product to create a new factor, $f2(B, E)$, which is a function of B and E:

$$f2(B, E) = \sum_A [P(A \mid B, E) * f1(A)]$$

For example, to compute $f2(\text{true}, \text{true})$ (for $B=\text{true}$, $E=\text{true}$):

$$f2(\text{true}, \text{true}) = [P(A=\text{true} \mid B=\text{true}, E=\text{true}) * f1(\text{true})] + [P(A=\text{false} \mid B=\text{true}, E=\text{true}) * f1(\text{false})]$$

If $P(A=\text{true} \mid B=\text{true}, E=\text{true}) = 0.95$ and $P(A=\text{false} \mid B=\text{true}, E=\text{true}) = 0.05$:

$$f2(\text{true}, \text{true}) = (0.95 * 0.63) + (0.05 * 0.0005) = 0.5985 + 0.000025 = 0.598525.$$

This process is repeated for all combinations of B and E (e.g., B=true, E=false; B=false, E=true; B=false, E=false), resulting in a table for $f_2(B, E)$.

Step 3: Eliminating 'E'

Now, we eliminate the hidden variable 'E'. The factors involving 'E' are $P(E)$ and $f_2(B, E)$.

We join them: $P(E) * f_2(B, E)$.

Then, we sum out 'E' to create a new factor, $f_3(B)$, which is a function of B:

$$f_3(B) = \sum_E [P(E) * f_2(B, E)]$$

For example, to compute $f_3(\text{true})$ (for B=true):

$$f_3(\text{true}) = [P(E=\text{true}) * f_2(\text{true}, \text{true})] + [P(E=\text{false}) * f_2(\text{true}, \text{false})]$$

If $P(E=\text{true}) = 0.002$ and $P(E=\text{false}) = 0.998$, and assuming $f_2(\text{true}, \text{false})$ was computed as 0.183:

$$f_3(\text{true}) = (0.002 * 0.598525) + (0.998 * 0.183) = 0.00119705 + 0.182634 = 0.18383105.$$

This is repeated for B=false to get $f_3(\text{false})$. The resulting $f_3(B)$ is proportional to $P(B, j, m)$.

Step 4: Normalization

The final factor $f_3(B)$ gives us values proportional to $P(B=\text{true}, j, m)$ and $P(B=\text{false}, j, m)$. To get the actual posterior probabilities $P(B | j, m)$, we need to normalize these values so they sum to 1.

Normalization constant (α) = $f_3(\text{true}) + f_3(\text{false})$.

$$P(B=\text{true} | j, m) = f_3(\text{true}) / \alpha$$

$$P(B=\text{false} | j, m) = f_3(\text{false}) / \alpha$$

For instance, if $f_3(\text{true}) = 0.0019$ and $f_3(\text{false}) = 0.0045$, then $\alpha = 0.0019 + 0.0045 = 0.0064$.

$$P(B=\text{true} | j, m) = 0.0019 / 0.0064 \approx 0.296875$$

$$P(B=\text{false} | j, m) = 0.0045 / 0.0064 \approx 0.703125$$

This final step yields the desired posterior probabilities.

3. Computational Complexity and Ordering

The efficiency of variable elimination is not determined by the total number of variables (N) in the network, but rather by the size of the largest intermediate factor created during the elimination process. If the largest factor involves 'k' variables, the complexity is exponential in 'k' (e.g., 2^k). In contrast, a naive full joint distribution approach would be exponential in N (2^N). For sparse networks, 'k' can be significantly smaller than N, leading to substantial computational savings.

The order in which variables are eliminated profoundly impacts the size of these intermediate factors. A poorly chosen elimination order can lead to large factors, effectively collapsing the benefits of the algorithm and making it as inefficient as

computing the full joint distribution (worst case 2^N). Conversely, an optimal elimination order can keep factor sizes small, making the inference tractable.

However, finding the optimal elimination order is an NP-hard problem. This means that for large or complex networks, finding the absolute best order is computationally infeasible. In practice, heuristics are employed to find good, though not necessarily optimal, elimination orders. Examples of such heuristics include eliminating variables that minimize the size of the next factor or eliminating variables with the fewest neighbors.

Beyond exact inference, Bayesian networks have connections to other complex computational problems. For instance, they can be used to solve the Boolean satisfiability problem (SAT) and even count the number of solutions to a SAT formula, a problem known as model counting, which belongs to the #P-complete complexity class. This demonstrates that Bayesian networks can tackle problems that are inherently harder than NP-complete problems. For specific network structures, like tree-structured Bayesian networks, inference can be performed in polynomial time, highlighting the importance of network topology.

Key Takeaways

- [] Inference in Bayesian networks involves computing posterior probabilities of query variables given evidence.
- [] The conditional probability $P(Z | E)$ is proportional to $P(Z, E)$, allowing for normalization at the end.
- [] Hidden variables must be summed out (marginalized) from the joint distribution during inference.
- [] Variable elimination is an exact inference algorithm that leverages dynamic programming to avoid redundant computations.
- [] The algorithm works by iteratively joining factors and summing out variables.
- [] The computational complexity of variable elimination is exponential in the size of the largest intermediate factor created.
- [] The order of variable elimination is critical for efficiency, but finding the optimal order is an NP-hard problem.
- [] Bayesian networks can be used to solve problems in higher complexity classes, such as #P-complete model counting.
- [] For specific network structures, like tree-structured Bayesian networks, inference can be polynomial time.

Conclusion

Variable elimination stands as a cornerstone algorithm for exact inference in Bayesian networks, offering a structured and efficient method to navigate the complexities of probabilistic reasoning. By systematically breaking down the problem into manageable factor operations and leveraging the network's topology, it allows us to compute posterior probabilities that would be intractable with a brute-force approach. This capability is fundamental for AI systems that must make decisions under uncertainty, from medical diagnosis to autonomous navigation.

While variable elimination provides exact answers, its computational cost can still be significant for very dense or large networks, especially given the NP-hard nature of finding an optimal elimination order. This limitation often necessitates the use of heuristic ordering strategies or, for even larger problems, approximate inference algorithms. Nevertheless, understanding variable elimination provides a deep insight into the mechanics of probabilistic graphical models and their profound connection to various computational challenges, including those in higher complexity classes. It serves as a vital stepping stone towards appreciating the broader landscape of AI's approach to uncertainty.